

Cryptographie

Vous allez jouer le rôle d'un espion qui intercepte des messages secrets et tente de les déchiffrer.

Cours 1 - Chiffre de César.

Le **chiffre de César** est une façon simple de coder un message afin de conserver le secret du contenu jusqu'à son destinataire. Il s'agit tout simplement de décaler chaque lettre du message.

Voyons l'exemple d'un décalage de trois lettres : A devient D ; B devient E ; etc.

ABCDEF~~GHIJKLMNOP~~QRSTUVWXYZ
DEFGHIJKLMNOPQR~~STUVWXYZ~~ABC

lettres du message en clair
lettres du message codé

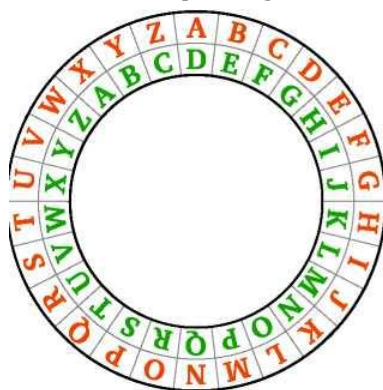
Par exemple, le message :

CAPTUREZ IDEFIX

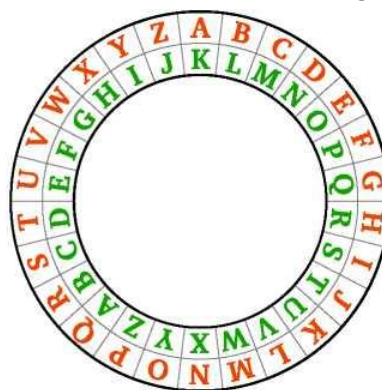
se chiffre en :

FDSWXUHC LGHILA

Une autre façon de présenter le décalage est de placer les alphabets en clair (anneau extérieur en rouge) et codé (anneau intérieur en vert) sur deux roues concentriques. À gauche un décalage avec $k = 3$ et à droite un décalage avec $k = 10$.



Décalage $k = 3$



Décalage $k = 10$

Pour chiffrer des messages on passe des lettres rouges à l'extérieur aux lettres vertes à l'intérieur.
Pour déchiffrer des messages il suffit de faire l'opération inverse : passer des lettres vertes aux lettres rouges !

Question 1 :

Déchiffrez, à la main, les messages suivants :

- EORTXHC DVWHULA chiffré avec un décalage $k = 3$.
- YE OCD ZXXYBKWSH chiffré avec un décalage $k = 10$

Cours 2 - Passer d'un caractère à un nombre et inversement.

On préfère travailler avec des nombres qu'avec des lettres !

- **Alphabet** : Il s'agit d'une variable globale, de type `str`, définie en dehors de toute fonction, et contenant toutes les lettres de l'alphabet, en majuscules :

`Alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"`

Les variables définies dans l'espace global du script, en dehors de toute fonction sont appelées des variables globales. Ces variables sont accessibles à travers l'ensemble du script et accessible en seulement en lecture à l'intérieur des fonctions utilisées dans ce script.

- **Numérotation**. On numérote chaque lettre : 0 pour A, 1 pour B, 2 pour C, ..., 25 pour Z.

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25

- **D'un numéro vers une lettre.**

Pour récupérer la lettre correspondant au rang i , il suffit d'écrire l'instruction `Alphabet[i]`.

Par exemple `Alphabet[2]` vaut 'C'.

- **D'une lettre vers son numéro.**

Pour récupérer le rang d'une lettre, notée `car`, il suffit d'écrire : `Alphabet.index(car)`

Par exemple `Alphabet.index('D')` vaut 3.

- **César.** Le chiffrement de César de décalage k correspond juste à une addition : le numéro j du caractère chiffré est égal au numéro i du caractère en clair auquel on ajoute le décalage k , le tout modulo 26 :

$$j = (i + k) \% 26$$

Par exemple, avec un décalage $k = 3$:

- le caractère numéro 0 (**A**) est chiffré en le caractère numéro $0 + 3 = 3$ (**D**),
- le caractère numéro 1 (**B**) est chiffré en le caractère numéro $1 + 3 = 4$ (**E**),
- ...
- le caractère numéro 25 (**Z**) est chiffré en le caractère numéro $25 + 3 = 28$ qui vaut 2 modulo 26 (c'est donc bien **C**)

Activité 1 - Chiffre de César.

Objectifs : programmer le chiffrement et le déchiffrement du chiffre de César.

1. Programmez une fonction `chiffre_cesar_caractere` qui prend pour paramètres un caractère `car` et un entier k , et qui renvoie le caractère de l'alphabet situé k rangs après (modulo 26).

Indications

- Récupérez le rang i du caractère à chiffrer dans `Alphabet`.
- Ajoutez k modulo 26 par la formule : $j = (i + k) \% 26$.
- Renvoyez le caractère correspondant au rang j de `Alphabet`.

Exemple : avec un décalage de $k = 3$, **A** devient **D** et **Z** devient **C**.

2. Programmez une fonction `chiffre_cesar_phrase` qui prend pour paramètres une chaîne de caractères `phrase` et un entier k , et qui renvoie la phrase codée par un décalage de César de k .

Indication : si un caractère n'est pas une lettre majuscule, alors conservez-le tel quel dans la phrase chiffrée.

Le test se fait par :

```
if car not in Alphabet:
```

Exemple : avec un décalage de $k = 3$, **CAPTUREZ IDEFIX !** devient **FDSWXUHC LGHILA !**

3. Programmez une fonction `dechiffre_cesar_phrase` qui prend pour paramètres un entier k et une chaîne de caractères `phrase` chiffrée par un décalage de César de k , et qui renvoie la phrase en clair (phrase déchiffrée).
4. On se place dans la peau d'un espion qui a intercepté un message chiffré par un chiffrement de César, mais qui ne connaît pas la clé k . Programmez une fonction `attaque_cesar` qui prend pour paramètre une chaîne de caractères `phrase` et qui affiche les déchiffrements, en testant toutes les clés k possibles.
5. Vous avez intercepté le message envoyé par le camp Babaorum à Jules César :
HSFGJSEAP F S HDMK VW HGLAGF
Quelle est la clé de ce chiffrement ?

Cours 3 - Chiffrement par substitution

Le chiffre de César est trop facile à attaquer, même si on ne connaît pas le décalage. Pour compliquer la tâche d'un espion, on introduit le **chiffrement par substitution**. À chaque lettre de l'alphabet en clair (ici en rouge, en majuscules), on associe une lettre au hasard (ici en vert, en minuscules). Voici un exemple :

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
y	k	c	o	d	m	f	j	g	z	a	x	r	n	b	u	t	q	i	p	h	w	e	s	v	l

- **Chiffrement.** On remplace chaque caractère du message par sa lettre substituée. Le message **BONJOUR** devient **kbnzbhq**.
Exercice : chiffrez la phrase **AU REVOIR**.

- **Déchiffrement.** On fait l'opération inverse : **kbnkbn** donne **BONBON**.

Exercice : déchiffrez **ywd cdiyq**.

Bien évidemment, pour les lettres d'arrivée, on aurait pu choisir un autre mélange.

L'expéditeur et le destinataire du message doivent au préalable se mettre d'accord sur la substitution choisie.

Activité 2 - Chiffrement par substitution.

Objectifs : programmer le codage et le décodage d'un chiffrement par substitution.

On se donne une substitution par un alphabet de départ suivi de son remplacement.

On prendra l'exemple suivant, défini par deux variables globales :

```
Alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
```

```
Melange = "ykcodmfjgzaxrnbutqiphwesvl"
```

1. Programmez une fonction `chiffre_substitution_caractere` qui prend pour paramètre un caractère `car` et qui renvoie le caractère chiffré avec la substitution choisie.

Indications.

- Récupérer le rang i du caractère à chiffrer dans `Alphabet`.
- Renvoyer le caractère correspondant au rang i de `Melange`.

Exemples : A devient y et B devient k.

2. Programmez une fonction `chiffre_substitution_phrase` qui prend pour paramètre une chaîne de caractères `phrase` et qui renvoie la phrase codée par substitution.

Exemple. Après substitution **PAS DE POTION POUR OBELIX !** devient **uyi od ubpgbn ubhq bkd xgs !**

3. Programmez une fonction `dechiffre_substitution_phrase` qui prend pour paramètre une chaîne de caractères `phrase` chiffrée par substitution et qui renvoie la phrase en clair.

Indications : vous pourrez commencer par écrire une fonction `dechiffre_substitution_caractere` qui se contente de déchiffrer un seul caractère.

Cours 4 - Attaque statistique

Pour un espion qui intercepterait un message codé sans connaître la substitution choisie, il n'est plus possible de tester toutes les possibilités.

En effet, il y a $26 \times 25 \times 24 \times \dots \times 2 \times 1$ choix possibles pour l'alphabet mélangé, ce qui fait environ 4×10^{26} clés !

Statistiques

Pour un texte assez long, les lettres n'apparaissent pas toutes avec la même fréquence.

En français, les lettres les plus rencontrées sont dans l'ordre :

ESAINTRULDCPMVQGFHBXJYZKW

La **fréquence d'apparition d'une lettre**, en pourcentage, est donnée par la formule :

$$\text{fréquence d'apparition d'une lettre} = \frac{\text{nombre d'occurrences de la lettre}}{\text{nombre total de lettres}}$$

Dans un texte en français, les fréquences sont proches de :

E	S	A	I	N	T	R	U	L	O	D
14.69 %	8.01 %	7.54 %	7.18 %	6.89 %	6.88 %	6.49 %	6.12 %	5.63 %	5.29 %	3.66 %

Attaque

On a intercepté un message, mais on ne connaît pas la substitution.

Comment utiliser les statistiques pour décrypter le message ?

Voici une méthode d'attaque : dans le texte chiffré, on cherche la lettre qui apparaît le plus et, si le texte est assez long, cela devrait être le chiffrement du **E** ; la lettre qui apparaît ensuite dans l'étude des fréquences devrait être le chiffrement du **S**, puis le chiffrement du **A**, **I**, **N**, **T**... On obtient ainsi un déchiffrement partiel du message, sous la forme d'un texte à trous et il faut ensuite deviner les lettres manquantes.

Un exemple

Par exemple, déchiffrons la phrase :

jm dw ug jddbhbwm y jmlj cjtltjdjd

On compte les apparitions des lettres :

j : 7 d : 5 m : 3 b, l, t, w : 2

On suppose donc que le **j** crypte la lettre **E**, le **d** la lettre **S**, ce qui donne :

E * S * * * ESS * * * * * E * * E * * * E * SES

Ensuite, la lettre qui apparaît le plus est le **m**.

D'après les fréquences, elle devrait correspondre à **A**, **I**, **N** ou **T**.

Ainsi le premier mot serait **EA**, **EI**, **EN** ou **ET** : seuls les deux derniers sont des mots valides.

- Si **m** → **N**, la phrase se déchiffre en :

EN S * * * ESS * * * * N * EN * E * * * E * SES

ce qui n'est pas très clair,

- Si **m** → **T**, c'est mieux !

ET S * * * ESS * * * * T * ET * E * * * E * SES

En cherchant où placer les lettres les plus fréquentes suivantes (**A**, **I**, **N**) puis les autres, avec un peu de patience et de bon sens, on décrypte le message :

ET SI ON ESSAYAIT D ETRE HEUREUSES

Activité 3 - Attaque statistique

Objectifs : utiliser la fréquence d'apparition des lettres pour décrypter un message en essayant de deviner la substitution.

1. Substitution.

Programmez une fonction `substitution` telle que l'appel

```
substitution(phrase, Alphabet_depart, Alphabet_arrivee)
```

substitue, dans `phrase`, les lettres de `Alphabet_depart` par celles de `Alphabet_arrivee`.

Indications

- C'est presque la même fonction que l'activité précédente.
- La fonction `substitution` doit permettre un déchiffrement partiel d'une phrase. Voir l'exemple ci-dessous.

Exemple : utilisation pour un décryptage partiel

- On veut décrypter :

jdolwt jd tm vb ?

- Admettons que l'on ait identifié que **j** codait **E**, **d** codait **S**, **m** codait **T**.
- On définit donc un alphabet de départ et celui (partiel) d'arrivée :

```
Alphabet_depart = "abcdefghijklmnopqrstuvwxy"
```

```
Alphabet_arrivee = "...S....E..T....."
```

- Alors avec `phrase = "jdolwm jd mt vb ?"`, l'instruction

```
substitution(phrase, Alphabet_depart, Alphabet_arrivee)
```

renvoie :

"ES...T ES T. .. ?"

- Il reste à deviner **ESPRIT ES TU LA ?**

2. Statistiques.

Programmez une fonction `statistiques` qui prend pour paramètre une chaîne de caractères `phrase`, calcule et renvoie le nombre d'apparitions des lettres dans une phrase. Indications.

- On suppose que la phrase est écrite en minuscules, on ne tient pas compte des autres caractères.
- La fonction renvoie la liste des nombres correspondant à chaque lettre de l'alphabet. Par exemple `[3, 0, 2, ...]` signifie qu'il y a 3 lettres **a**, pas de lettres **b**, 2 lettres **c**, etc.

En déduire une fonction `affiche_statistiques` qui prend pour paramètre une chaîne de caractères `phrase` et affiche proprement les correspondances entre les lettres qui apparaissent dans la phrase et leurs nombres d'apparitions.

Fréquences.

Ecrire deux fonctions `fréquences` et `affiche_frequences` prenant toutes les deux pour paramètre une chaîne de caractères `phrase` et qui, respectivement, calculent et affichent les fréquences d'apparitions des lettres.

Indication.

Il est nécessaire, pour calculer les fréquences, de déterminer le nombre de lettres que contient la `phrase` (on ne tient compte que des lettres en minuscules). Par exemple, dans la phrase `jdolwt jd tm vb ?`, le nombre de lettres est 12.

La fréquence d'apparition du `j` est donc $\frac{2}{12} \approx 0,17$.

3. Énigmes. Vous laisserez visibles vos recherches dans le notebook en expliquant vos démarches dans des cellules de type Markdown.

Décrypter les trois citations suivantes. Chacune a été chiffrée par une substitution différente.

- Les frères Goncourt :

ay dmymndmnlxlv vdm ay shvjnvhv fvd dznvgzvd ngvcyzmvd

- Charles Darwin :

**apy pywpfpy tdlv ydjsvspng np ybng woy apy pywpfpy apy wady lbjgpy nv apy wady
vngpaavzpngpy movy fpaapy tdlv y okowgpng ap mvpdc ode fionzpmppngy**

- Albert Einstein :

**kw yjnzfcn, i nmy lqwpz zp mwcy yzqy ny lqn fenp pn azpiyczppn. kw ofwyclqn, i nmy lqwpz yzqy azpiyczppn ny
lqn onfmzppn pn mwcy ozqflqzc. cic, pzqm wszpm fhqpc yjnzfcn ny ofwyclqn : fenp pn azpiyczppn ny onfmzppn
pn mwcy ozqflqzc !**

Cours 5 - Chiffrement de Vigenère

Un des principaux défauts du chiffre de César (et du chiffrement par substitution) est qu'une lettre (par exemple **A**) est toujours chiffrée par la même lettre (par exemple **D**). Le chiffrement de Vigenère est une version améliorée du chiffre de César.

On regroupe d'abord les lettres de notre message par blocs, par exemple ici par blocs de longueur 3 :

IL ETAIT UNE FOIS

devient

ILE TAI TUN EFO IS

(les espaces sont purement indicatifs, dans la première phrase ils séparent les mots, dans la seconde ils séparent les blocs).

Si n est la longueur d'un bloc, alors on choisit une clé constituée de n nombres de 0 à 25 : $[k_1, k_2, \dots, k_n]$.

Le chiffrement consiste à effectuer un chiffrement de César, dont le décalage dépend du rang de la lettre dans le bloc :

- un décalage de k_1 pour la première lettre de chaque bloc,
- un décalage de k_2 pour la deuxième lettre de chaque bloc,
- ...
- un décalage de k_n pour la n -ème et dernière lettre de chaque bloc.

Pour notre exemple, si on choisit comme clé [4, 2, 3] alors pour le premier bloc « **ILE** » :

- un décalage de 4 pour **I** donne **M**,
- un décalage de 2 pour **L** donne **N**,
- un décalage de 3 pour **E** donne **H**,

Ainsi « **ILE** » devient « **MNH** ». On recommence avec le bloc « **TAI** » qui devient « **XCL** ». Le chiffrement complet donne :

MNH XCL XWQ IHR MU

autrement dit la phrase chiffrée est :

MN HXCLX WQI HRMU

Exercice

Chiffrez, à la main, la phrase **UNE PRINCESSE GEEK** avec le chiffrement de Vigenère et la même clé [4, 2, 3].

Déchiffrement.

Pour déchiffrer, il suffit d'inverser la clé, c'est-à-dire que c'est la même procédure que le chiffrement mais avec la clé $[-k_1, -k_2, \dots, -k_n]$. Par exemple l'opposé de la clé [4, 2, 3] est la clé [-4, -2, -3].

Comme on travaille modulo 26, cette dernière clé vaut aussi [22, 24, 23].

Exercice

Déchiffrez, à la main, la phrase **QKOSW HX VLRVLR**, chiffrée avec la même clé [4, 2, 3].

Activité 4 - Chiffrement de Vigenère

Objectifs : programmer le chiffrement de Vigenère et éventuellement trouver une attaque.

1) Programme une fonction `chiffre_vigenere(phrase, cle)` qui chiffre la phrase donnée selon le chiffrement de Vigenère, pour la clé donnée sous la forme d'une liste de décalages, `cle = [k1, k2, ..., kn]`.

Exemple.

La phrase **AAA ABC** avec la clé [1, 2, 3] renvoie la phrase **BCD BDF**.

Indications.

- Il faut décaler chaque lettre selon un décalage k_i comme pour le chiffre de César.
- Pour savoir quel indice i convient, on pourra utiliser un compteur i , initialisé à 0, auquel on ajoute 1 à chaque lettre de l'alphabet rencontrée, compteur qui prend ses valeurs modulo n (qui est la longueur de la clé).

2) Attaque du chiffrement de Vigenère. A faire sur une feuille de classeur et à rendre.

Décryptez le message suivant et trouvez son auteur : ce message a été codé par un chiffrement de Vigenère avec une clé (inconnue !) de longueur 4.

**DL ZHGIVUEL OD UL LQK TYDVL OIL XEU DLC YEIOSASOI K VXJ K BBI WA PYWYC T WBH
QDVBIIBO BWZ QUFZ SDLLVBANODLZ CEEA OCHSSI VL NEMAOI**

Indications page suivante...

Indications

DL ZHG VUEL OD UL LQK TYDVL OIL XEU DLC YEIOSASOI KVXJ KBBI WA PYWYC T WBH
QDVBIIBO BWZ QUFZ SDLLVBANODLZ CEEA OCHSSI VL NEMA OI

1) Pour trouver la valeur du premier décalage, on prend les lettres de rangs 1, 1 + longueur_clé, 1+2 longueur_clé, etc. : toutes ces lettres sont codées avec le même décalage.

phrase1 = dgeukvldeakkwwwdiwflalehvm

On lance ensuite une analyse de fréquences sur cette phrase :

- quelle est la lettre la plus fréquente ?
- que peut-on en déduire ?

2) En s'appuyant sur le raisonnement de la question 1, faites une proposition de clé : cle = [... , ... , ... , ...].

Vous expliquerez les détails de votre démarche.

3) Essayez de déchiffrer le message avec cette clé : le résultat est-il satisfaisant ? Argumentez votre réponse puis faites les modifications nécessaires pour réussir à déchiffrer le message.